

CS380: Math for AI; Homework #1: The Vector

Benjamin Byington Belden
hw1-Belden-26fa380

September 2026

Due Monday, September 14 at 11:59pm Central

Please read the following independent completion attestation before starting the homework. Then digitally sign it, acknowledging that you understand the guidelines for this homework, and have followed them.

I declare that I have completed this assignment entirely on my own, and following the generative AI policies. I have read the UAB Academic Integrity Code and understand that any breach of this code may result in severe penalties, including failure of the class.

Signature: *Ben Belden*

Q1: The Hyperplane (written)

In this question, you will prove one of the key steps in moving from the algebra of a hyperplane to the geometry of a hyperplane: the coefficient vector of the linear equation is the normal.

Lemma: Let a be a vector in d -space, and b a scalar.

The normal of the hyperplane defined by the linear equation $a \cdot x = b$ is a .

NOTE: $a \perp x \Leftrightarrow a \cdot x = 0$

Proof.

$$a = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_d \end{pmatrix} \quad x = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_d \end{pmatrix}$$

$$a \cdot x = \sum_{i=1}^d a_i * x_i = a_1 x_1 + a_2 x_2 + a_3 x_3 + \cdots + a_d x_d = b$$

$$H = \{x \in \mathbb{R} : a \cdot x = b\}$$

$$x_1, x_2 \in H \rightarrow a \cdot x_1 = b \wedge a \cdot x_2 = b$$

$$(a \cdot x_1) - (a \cdot x_2) = b - b = 0$$

$$a \cdot (x_1 - x_2) = 0 ; x_1 - x_2 \parallel H$$

$$a \perp (x_1 - x_2)$$

■

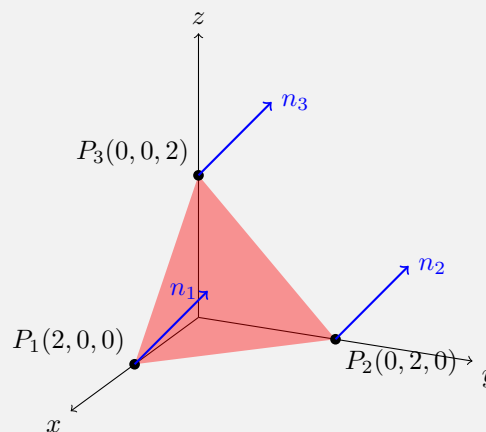
An oriented hyperplane is a hyperplane where one halfspace is positive and the other halfspace is negative, with the normal pointing towards the positive halfspace. So flipping the normal yields a different oriented hyperplane. One geometric representation for an oriented hyperplane is by a unit normal (pointing in the direction of the positive hyperplane) and a point P on the hyperplane.

- Find a problem with this representation.
- Explain the problem.
- Find a geometric representation of an oriented hyperplane that solves this problem, (hint: keep the normal, change the point to something else).
- Find another advantage of this new representation (aside from solving the old problem).

→ an oriented hyperplane can be represented by $(\text{unit normal}, P)$

- n = normal vector
- P = any point on the hyperplane

suppose: $x + y + z = 2$



$$n = (1, 1, 1) \rightarrow n_1 = (t + 2, t, t), n_2 = (t, t + 2, t), n_3 = (t, t, t + 2)$$

$$\text{unit normal} = (1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})$$

Because the hyperplane $x + y + z = 2$ is a completely flat surface, its orientation never changes. The normal vector remains exactly the same at every single point on the plane.

In the situation above → all three points (P_1, P_2, P_3) grouped with the unit normal vector $(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})$ give the same oriented hyperplane $(\text{unit normal}, P_n)$, this is our problem.

A hyperplane can be defined by a point P and a normal n . We can choose from infinitely many P (as long as the point's location is on the hyperplane), if we pick a point P_1 on the plane, the pair (n, P_1) describes the hyperplane. If we slide along the plane and pick a different point P_2 , the pair (n, P_2) describes the exact same hyperplane. So, our choice of the point P is not unique.

Why is this an issue? I believe it would be easier to explain with real world situations. Honestly, it took myself some time to understand this error, "Of course, if you were to pick two different points that lie on the same flat surface and computed their respected unit normal vectors, they will correspond to the same hyperplane, what's the issue?" Because n stays exactly the same while you pick different points, you can generate an infinite number of pairs describing the same hyperplane.

Ideally, every unique geometric hyperplane should map to exactly one unique mathematical representation. How is this possible? We can eliminate redundancy with the unit normal vector by curving the surface. If we turn the flat surface into a curved one, every point P on our curved surface can correspond to a different unit normal vector n . But, by definition, a hyperplane is a flat geometric shape and a flat geometric shape's normal vectors will always be identical no matter where you are on the shape. Okay, no-go for curving the surface.

Maybe the issue is not with the hyperplane itself, but the way we are describing it $\rightarrow (n, P)$. From previous prerequisite knowledge (n, P) seemed like a perfectly reasonable way to describe a hyperplane $\rightarrow$ if you pick a second point P_0 , create a vector (lets call it vector v) by subtracting P from P_0 , take the inner product between our new vector v and our normal vector n and, finally, if this inner product outputs 0, we know the angle between v and n is 90° , and the point P_0 will lie on the hyperplane. Easy stuff. But, what if not a mammal is deducing our (n, P) but a computer?

Imagine you want to tell a computer exactly where a flat wall is located in a room:

- The Normal Vector: Tells the computer which way the wall is facing (West). Since the wall is perfectly flat, it faces West no matter where you stand on it.
- The Point: Tells the computer, "The wall faces West, and it passes through this specific particle (Imagine a particle as a static entity. Of course, we know this not to be true, as they are in constant motion.)."

The issue is that you could pick a particle on the far left, or a particle on the far right. The wall will not move or curve, but your description changes pending on which particle you choose.

If the hyperplane curved, you would need a different normal vector for every point, which makes the computation overhead too high and the math super complex. We want/need hyperplanes to be flat because their flatness allows faster computation time, higher optimization, and better efficiency. Instead of forcing the hyperplane to curve, we can change how we describe its position:

- Issued Interpretation: "The plane faces direction n and passes through the point P ."
- Fixed Interpretation: "The plane faces direction n and is exactly d units away from the origin $(0, 0, 0)$."

We stop using an arbitrary point on the plane. Rather, we use a single scalar number $\rightarrow$ the orthogonal shortest distance from the origin to the hyperplane.

Instead of storing the combo (n, P) , we store the pair (n, d) , where d is a scalar calculated by taking the inner product of the normal vector from the plane and any point lying on the hyperplane:

$$d = n \cdot P$$

From our above question; "Find a geometric representation of an oriented hyperplane that solves this problem" ... $n \cdot P$ is not a geometric description but a mathematical description, so, although our problem of infinite representation has been solved, our specific question above (geometric) has not.

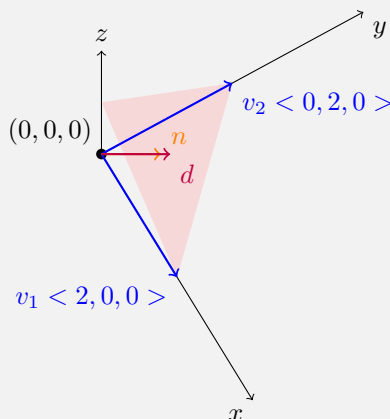
What is a geometric interpretation of something? According to the oh so reliable Google a geometric representation is; "a way to look at a math formula, number, or rule as a visual shape, graph, or picture instead of just a set of text and number." If the mathematical representation of an inner product is $u \cdot v$, we know that its corresponding geometric representation is $\|u\|\|v\|\cos\theta$. So, with this being stated, our new (geometric) representation is:

$$d = \|n\|\|P\|\cos\theta$$

$\|n\|$ and $\|P\|$ represent the geometric lengths of our vector P (the vector starting at the origin $(0, 0, 0)$ and ending at the user-picked point P) and our vector n (the vector starting at the origin and extending a

magnitude of 1 in the direction of the computed normal coordinates). The θ represents the angle between these two vectors, and we take the cos because, well, adjacent over hypotenuse (draw a right triangle).

Now, let us return to our hyperplane above: $x + y + z = 2$



$$P(v_1) \cdot n = \langle 2, 0, 0 \rangle \cdot \langle 1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3} \rangle = 2/\sqrt{3} = d$$

$$P(v_2) \cdot n = \langle 0, 2, 0 \rangle \cdot \langle 1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3} \rangle = 2/\sqrt{3} = d$$

- Draw a line from the origin to your point P .
- Project that line down onto the directional track of the normal vector n .
- The resulting length along that track is your scalar d .

Why does this eliminate repetition?

By switching from a point P to the scalar distance d :

- Only One Unique Distance Will Exist: A flat plane (hyperplane) only has one closest point to the origin $\rightarrow$ there is only one unique value for d .
- A 1-to-1 Map: Every unique pair (n, d) now corresponds to exactly one unique oriented hyperplane.

"But, the point $P(v_1)$ and the point $P(v_2)$ both correspond to $(\langle 1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3} \rangle, 2/\sqrt{3})$ if we were to showcase it using our new mathematical representation (n, d) , isn't this redundant?"

Yes, geometrically the points $(2, 0, 0)$ and $(0, 2, 0)$ belong to the exact same hyperplane. However, as data coordinates, the numbers themselves are different. We have returned back to our massive issue! If you use the (n, P) description, a computer looks at the pair (n, P_1) and the pair (n, P_2) and sees two different coordinates of inputs. It had to do extra math just to realize they describe the same hyperplane $\rightarrow$ overly complex, not scalable, and redundant!

But, by computing the inner product, both of those different coordinate pairs project into the exact same distance along the normal vector. This forces every point on the plane to project onto the exact same spot on the normal vector line $\rightarrow$ only one unique value for d eliminating the repetition and reducing our computation complexity.

Q2: The Word Vector (written and code)

To prepare for this question, download the glove.2024.wikigiga.50d dataset from Stanford GloVe website, extract it, and rename it 'glove.2024.wikigiga.50d.txt'. You will use this in your experiments.

For simplicity, you will use naive algorithms to work with the vectors in this homework, since our focus is on learning basic linear algebra of vectors $\rightarrow$ contemplate more sophisticated algorithms for future.

Q2a: Extraction of the Word Vectors

You will start by extracting the following word vectors from glove.2024.wikigiga.50d.txt:

good, evil, devil, angel, goods, cup, happy, ethical

Store in a matrix, one word vector per column. (We call this a column-major data matrix.)

The shape of the matrix will be (50, 8).

Use the order of the words, so the 4th column will be the word vector for 'angel'.

To solve this problem, write a Python function in your script that extracts a word into a NumPy vector: input is the path to the dataset (a string) and the word (a string);

output is the word vector (NumPy array).

A typical call would be ('glove.2024.wikigiga.50d.txt', 'good') Use this code to build the matrix and report it below.

NOTE: until we cover matrices, you could temporarily store in 8 separate NumPy vectors.

Figure 1: 8 <50, 1> Column Word Vectors

```
In [6]: %runfile /Users/benbelden/Desktop/UAB/fall2026/cs380/hw01/hw1.belden.26fa380.py --wdir
-0.3690 -0.6987 -0.6924 -0.7837 0.5146 0.6096 -0.4105 -0.0678
-0.2872 0.6481 -0.0015 0.5329 -0.5081 -0.6844 0.0426 0.9349
-0.5530 0.6437 0.1096 -0.4457 -0.7463 -0.8218 -0.4724 -0.7132
0.3667 -0.2295 -0.4868 -0.5894 0.3800 -1.1888 0.1186 0.3347
0.2809 0.3753 0.4815 0.1500 1.0555 0.8855 1.0463 -0.8715
-0.4085 0.4267 -0.4521 0.3921 0.5724 0.5271 -0.7597 0.6641
-0.0910 0.4268 0.1399 0.1119 1.3477 -0.5635 -0.3877 -0.0862
0.0807 1.2113 0.8158 0.6712 -1.4616 0.8038 0.3877 -0.4733
0.0232 0.6366 0.7845 0.6736 -0.4013 0.0949 0.2666 0.3087
-0.4835 -1.5247 -0.5840 -0.3916 0.1553 -1.1525 -0.3867 -0.1359
-0.8074 0.8086 -0.4460 0.2010 -0.2243 -0.7148 -0.5103 -0.1572
-0.3915 -0.4963 -0.0153 -0.3127 0.9975 0.4423 -0.3608 -1.0215
0.1477 -0.3466 0.0415 -0.0444 0.7363 0.6144 0.1033 -0.4970
-0.6107 0.2412 0.7432 -0.0398 1.6997 -0.7368 -0.8695 -0.8822
-0.5199 0.7695 -0.0062 -0.8678 0.1274 -1.3158 -0.2937 0.8724
0.3091 -0.9771 -0.0720 0.4387 0.1271 -1.9002 0.3449 -0.0837
-0.1556 0.2341 0.3027 0.5928 -0.1696 0.8392 0.0758 -0.8727
-0.4163 -0.6056 -0.7469 -0.3615 0.9365 0.4331 -0.7842 -0.6648
0.3578 0.6649 -0.1409 -0.2122 -0.2100 1.0981 0.2707 -0.0619
0.2759 0.3867 1.1020 1.3662 -1.0260 -0.0695 0.2169 -1.2158
-1.0457 -0.8746 -0.8629 0.5661 0.1188 -0.3975 -0.0701 -1.1192
0.0610 0.5740 0.4045 0.4678 -0.7525 -1.1838 0.2308 -0.3569
0.2819 -0.2538 0.3760 -0.2246 -0.2434 0.4887 0.6585 -0.3919
0.3100 0.1734 0.6816 -0.2521 -0.0688 -0.7993 0.1838 -0.1311
0.0619 -0.1044 -0.4806 -0.4544 0.3753 -0.7937 -0.9684 0.3059
0.0472 -0.4239 0.3306 0.3918 -0.1849 0.4325 -0.1212 0.4362
-0.3664 -0.1136 -0.5193 0.7024 -1.0233 -0.1027 -0.5907 0.0415
0.1228 1.3912 1.1243 1.8681 -0.9111 -0.4784 0.1590 0.1335
0.7665 0.3742 -0.7000 -0.5398 0.8906 -0.7047 0.4431 1.3873
-0.3900 -0.2100 0.5395 0.4664 -1.1756 -0.2483 -0.2820 -0.4065
0.0129 -0.7762 -0.5810 0.1296 1.1302 -0.3474 -0.0256 -0.5354
0.6010 -0.1160 0.4299 0.5459 -0.3802 1.0226 0.6930 -0.3207
-0.4631 -0.3962 -1.1201 -0.9054 0.0651 -0.0480 -0.5475 -0.6696
-0.2417 -0.4714 -0.3623 -0.0891 0.9652 1.8449 -0.3521 -1.2677
0.1221 -0.2275 -0.6178 0.1470 0.5241 -0.0831 -0.1493 -0.0991
-0.0733 0.0890 0.1587 0.7179 0.3375 -2.6067 0.0438 -0.0502
0.2569 0.7116 -0.3196 -0.4104 0.4888 -1.1685 0.5097 -0.2451
5.0116 2.7560 2.6096 2.5912 3.6288 4.3796 3.5184 2.4533
0.6706 0.4232 0.2816 0.4146 -0.2517 0.6597 0.3613 -0.1247
0.1679 0.3760 -0.2547 -0.8825 0.2019 -0.1820 0.1017 -0.4753
-0.1714 0.5738 0.4023 0.1067 -0.5022 0.2244 0.2437 0.1335
0.5593 0.0085 0.1950 0.8145 -0.3150 0.1497 0.8575 -0.7126
0.1836 -0.5939 -0.6839 -0.5274 0.3114 0.0436 0.5826 -0.7306
1.1993 1.4095 0.4883 -0.5995 1.2703 0.0897 1.0831 1.2103
-0.5844 -0.3092 0.1612 0.5996 1.0704 -0.4530 -0.2647 -0.7322
0.2487 0.2359 0.2371 0.1917 -0.4452 0.8052 -0.3827 1.3919
0.0316 0.6655 0.6334 0.5525 -0.0714 0.1438 -0.1499 -0.6934
-0.1940 1.3388 0.0990 -0.5046 -0.2864 -0.5209 -0.1347 1.0548
-0.1645 1.3197 0.6277 0.9628 0.4246 -0.0334 0.1263 0.4945
1.0036 0.8040 0.2332 0.4230 0.3237 0.4888 1.5879 -0.5982
```

SOME NOTES:

- IDE → spyder, python 3.14, hw1.belden.26fa380.py
- Upon completion of compilation to bytecode and execution via the Python Virtual Machine, all objects created/given values in source code are placed in an area called 'variable explorer' built into spyder.
- In 'variable explorer' you have ability to see a more accurate and immersive (50, 8) 'dataMatrix' created through the python source code logic.
- You can save this matrix as a '.spydata' file.
- Because '.spydata' files are something called a binary data archive containing a python/NumPy object and not readable Markdown text, the contents of the file will be hieroglyphic looking.
- The important parts within this binary data archive are a python pickle file and a NumPy array.
- The reason the file is unreadable is because a .spydata file views binary numerical data as if it were plain text.
- You can extract the actual (50, 8) matrix and show it as a readable table by opening it with a program that understands a binary format.
- .spydata file is an archive, extract the NumPy array (.npz) file from the .spydata file.
- To do this:
 - open terminal
 - cd into the directory containing the .spydata file
 - run 'tar -xvf matrixFilename.spydata' to extract
 - extracts .pickle and .npz
- .npz file is the file you want!
- Can use the .npz file now in source code to load and print the data

Q2b: Cosine Similarity

Next find the cosine similarity between each pair of word vectors, and store in a matrix of shape (8, 8). Report each similarity to 5 decimal places.

Use the above order: for example, the similarity between 'devil' and 'angel' would be stored in the entry in the 3rd row and 4th column, and the entry in the 4th row and 3rd column (hopefully they are the same!).

To solve this problem, write a Python function in your script that computes cosine similarity:

input is two word vectors (NumPy array, NumPy array);

output is the cosine similarity (NumPy scalar).

Write two versions of this function: using NumPy and computing it yourself using a for loop (see the template script).

How would you measure similarity between two word vectors?

a = good, b = evil, c = devil, d = angel, e = goods, f = cup, g = happy, h = ethical:

$$\begin{pmatrix} a/a & a/b & a/c & a/d & a/e & a/f & a/g & a/h \\ b/a & b/b & b/c & b/d & b/e & b/f & b/g & b/h \\ c/a & c/b & c/c & c/d & c/e & c/f & c/g & c/h \\ d/a & d/b & d/c & d/d & d/e & d/f & d/g & d/h \\ e/a & e/b & e/c & e/d & e/e & e/f & e/g & e/h \\ f/a & f/b & f/c & f/d & f/e & f/f & f/g & f/h \\ g/a & g/b & g/c & g/d & g/e & g/f & g/g & g/h \\ h/a & h/b & h/c & h/d & h/e & h/f & h/g & h/h \end{pmatrix}$$

Can you spot any patterns in the above (8, 8) template?

What's going to happen is: "cosine similarity measures the directional orientation of two vectors in a multi-dimensional space".

What does this mean? Cosine similarity is simply telling us how alike two vectors are in space.

$$\cos \theta = v \cdot w / \|v\| \|w\|$$

From our (8, 8) template above, the v is the letter in the numerator at each position and the w is the letter in the denominator.

If we think about the unit circle, cosine has a maximum value and a minimum value. Because of this restriction, the output from the above equation will be between the values of -1 and 1.

$$R: [-1, 1]$$

If you are wanting the angle θ between the two vectors v and w, take the inverse cosine of the right side of the equation.

The closer the output is to 1, the more similar.

The closer the output is to -1, the more different.

Figure 2: spyder Terminal cosSim Matrix

```
In [1]: %runfile /Users/benbelden/Desktop/UAB/fall2026/cs380/hw01/hw1.belden.26fa380.py --wdir
1.00000  0.56185  0.60906  0.46575  0.49144  0.57598  0.88230  0.52010
0.56185  1.00000  0.68816  0.45106  0.14713  0.28509  0.53960  0.45917
0.60906  0.68816  1.00000  0.70167  0.11696  0.39759  0.60395  0.24173
0.46575  0.45106  0.70167  1.00000  0.04855  0.32928  0.47432  0.09571
0.49144  0.14713  0.11696  0.04855  1.00000  0.32619  0.36205  0.20225
0.57598  0.28509  0.39759  0.32928  0.32619  1.00000  0.50082  0.15265
0.88230  0.53960  0.60395  0.47432  0.36205  0.50082  1.00000  0.30450
0.52010  0.45917  0.24173  0.09571  0.20225  0.15265  0.30450  1.00000
```

Can you pick up on any patterns yet?

Maybe it will be easier to see if we showcase the colored matrix spyder's variable explorer creates for its users?

Figure 3: spyder Variable Explorer cosSim Matrix

	0	1	2	3	4	5	6	7
0	1	0.56185	0.609062	0.465748	0.491436	0.575976	0.882303	0.520101
1	0.56185	1	0.688158	0.451059	0.14713	0.28509	0.539599	0.459173
2	0.609062	0.688158	1	0.701665	0.11696	0.397593	0.603954	0.241726
3	0.465748	0.451059	0.701665	1	0.048548	0.329279	0.474316	0.095713
4	0.491436	0.14713	0.11696	0.048548	1	0.326191	0.362052	0.202251
5	0.575976	0.28509	0.397593	0.329279	0.326191	1	0.500819	0.152653
6	0.882303	0.539599	0.603954	0.474316	0.362052	0.500819	1	0.304501
7	0.520101	0.459173	0.241726	0.095713	0.202251	0.152653	0.304501	1

The ones on the diagonal? These 'ones' represent two vectors being the exact same, so there similarity is 100% → you are taking the cosine similarity of a vector v with itself (a/a, b/b, c/c, ..., h/h).

Taking the ones out of account, which two words are:

most similar - entry (1, 7), (7, 1) $\rightarrow$ good and happy, happy and good
 least similar - entry (5, 4), (4, 5) $\rightarrow$ goods and angel, angel and goods

Q2c: Euclidean Length

Now find the length of our 8 word vectors, and store the results in a vector (again using the given order.)
 This output vector will be a vector in 8-space.
 Report each length to 5 decimal places.

To solve this problem, write a Python function in your script that computes the length of a vector in d-space:
 input is a vector in d-space (NumPy array);
 output is its Euclidean (2-norm) length (NumPy scalar).
 Write two versions of this function: using NumPy and computing it yourself using a for loop.

Figure 4: (1, 8) Euclidean Norms

```
In [1]: %runfile /Users/benbelden/Desktop/UAB/fall2026/cs380/hw01/hw1.belden.26fa380.py --wdir
5.90708 5.47518 4.56555 4.95621 6.19388 7.30059 5.05720 5.33920
```

NOTE:

Numpy - Manual: The dramatic performance speedup using NumPy 'items' versus building everything by hand is a result of NumPy operations running in highly optimized, compiled C code under the hood. Whereas python loops carry heavy overhead for type-checking and bytecode execution at every single iteration.

Q2d: Mikolov Example

One of the examples that (Mikolov 2013, 'Distributed Representations of Words and Phrases and their Compositionality' on arXiv) uses to illustrate that their word vectors behave properly (like vectors in linear algebra) is: $\text{vec}(\text{'King'}) - \text{vec}(\text{'Man'}) + \text{vec}(\text{'Woman'})$ is very close to $\text{vec}(\text{'Queen'})$.

In the GloVe 50-d dataset, what is the cosine similarity and Euclidean distance between these two vectors?
 Report to 5 decimal places.

Note that you would not expect as good results with these 50-dimensional vectors of GloVe as the 640-dimensional vectors of Mikolov.

Cosine Similarity $\text{vec}(\text{'King'}) - \text{vec}(\text{'Man'}) + \text{vec}(\text{'Woman'})$ with $\text{vec}(\text{'Queen'})$:

0.87155

Euclidean Distance $\langle \text{vec}(\text{'King'}) - \text{vec}(\text{'Man'}) + \text{vec}(\text{'Woman'}) \rangle$ and $\langle \text{vec}(\text{'Queen'}) \rangle$:

3.10763

Two Vectors:

$\text{vec}(\text{'King'}) - \text{vec}(\text{'Man'}) + \text{vec}(\text{'Woman'})$,

$\text{vec}(\text{'Queen'})$

Index	Math Vector	Queen Vector
Row 0	-0.80522	-1.46565
Row 1	1.04459	0.28975
Row 2	-0.20857	-0.20995
Row 3	-1.57146	-0.50926
Row 4	1.93464	1.65363
Row 5	-1.46446	-0.40026
Row 6	-1.00204	-0.35706
Row 7	0.68692	1.27760
Row 8	-0.43491	-0.15261
Row 9	-0.16137	-0.93582
Row 10	1.38474	0.71086
Row 11	0.21791	0.51987
Row 12	0.59524	0.50269
Row 13	-0.47586	-1.05627
Row 14	-0.60208	-0.78199
Row 15	0.01116	-0.01592
Row 16	-0.30610	0.13892
Row 17	0.29185	0.22256
Row 18	0.50935	0.38591
Row 19	0.13491	0.46453
Row 20	0.40268	0.18725
Row 21	0.51970	0.06171
Row 22	-1.02990	-0.46900
Row 23	0.37218	-0.30032
Row 24	-0.35430	-0.67780
Row 25	0.26668	0.22720
Row 26	-0.26461	-0.25464
Row 27	-0.49018	-0.25132
Row 28	-0.66991	-0.08804
Row 29	0.08989	0.04875
Row 30	-0.64997	-0.77613
Row 31	0.36550	0.66450
Row 32	0.77612	0.64523
Row 33	0.55191	1.00123
Row 34	-0.59623	-0.97132
Row 35	1.49567	0.81196
Row 36	-0.04088	0.03793
Row 37	3.56015	3.37402
Row 38	0.13204	0.29643
Row 39	0.35943	0.28261
Row 40	-0.44514	0.26219
Row 41	-1.07744	-1.21760
Row 42	0.14890	0.01475
Row 43	-0.05262	-0.16393
Row 44	0.17367	0.44624
Row 45	-0.16945	-0.62959
Row 46	-1.35451	-0.94544
Row 47	0.22413	0.04951
Row 48	1.39842	1.18359
Row 49	-0.29126	-0.23286

NOTE: All above computations take into account: vector ' $\langle \text{vec}(\text{'King'}) - \text{vec}(\text{'Man'}) + \text{vec}(\text{'Woman'}) \rangle$ ' and vector ' $\langle \text{vec}(\text{'Queen'}) \rangle$ ' are not unit vectors meaning their magnitudes are not one.

What to hand in

Generate a pdf of this document using Overleaf, naming the file 'hw1-LASTNAME-26fa380.pdf'.

Example: hw1-johnstone-26fa380.pdf

Write your code in a Python script named 'hw1-LASTNAME-26fa380.py'.

Example: hw1-johnstone-26fa380.py

I have given you a starting point in template-hw1-26fa380.py.

Please use this code as a starting point (e.g., use the same function names).

Zip the pdf and Python script into hw1-LASTNAME-26fa380.zip, and upload this one zip file to Canvas.